

✓ Syntecxhub AI Internship – Week 1

Project 1: Maze Solver using A* Search

Run the cells top to bottom (Runtime → Run all).

```
"""
Syntecxhub AI Internship - Week 1 - Project 1
Maze Solver using A* Search

Maze file format (plain text):
# wall
. open cell
S start
G goal
"""

import argparse
import heapq
import math
import sys
from typing import Dict, List, Optional, Set, Tuple

Cell = Tuple[int, int] # (row, col)

# -----
# 1. Maze representation: every open cell is a node, neighbours are edges
# -----
class Maze:
    def __init__(self, text: str):
        lines = [ln.rstrip("\n") for ln in text.strip("\n").splitlines() if ln.strip()]
        if not lines:
            raise ValueError("Maze is empty.")
        width = max(len(ln) for ln in lines)
        self.grid: List[str] = [ln.ljust(width, "#") for ln in lines]
        self.rows, self.cols = len(self.grid), width
        self.start: Optional[Cell] = None
        self.goal: Optional[Cell] = None

        for r, row in enumerate(self.grid):
            for c, ch in enumerate(row):
                if ch == "S":
                    self.start = (r, c)
                elif ch == "G":
                    self.goal = (r, c)
        if self.start is None or self.goal is None:
            raise ValueError("Maze must contain exactly one 'S' and one 'G'.")

    def is_open(self, cell: Cell) -> bool:
        r, c = cell
        return 0 <= r < self.rows and 0 <= c < self.cols and self.grid[r][c] != "#"

    def neighbors(self, cell: Cell) -> List[Cell]:
        """4-directional moves (up, down, left, right), each with cost 1."""
        r, c = cell
        candidates = [(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)]
        return [n for n in candidates if self.is_open(n)]

    @classmethod
    def from_file(cls, path: str) -> "Maze":
        with open(path, "r", encoding="utf-8") as f:
            return cls(f.read())

# -----
# 2. Heuristics (both admissible for 4-directional, unit-cost movement)
# -----
def manhattan(a: Cell, b: Cell) -> float:
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean(a: Cell, b: Cell) -> float:
    return math.hypot(a[0] - b[0], a[1] - b[1])

HEURISTICS = {"manhattan": manhattan, "euclidean": euclidean}
```

```

# -----
# 3. A* search
# -----
class SearchResult:
    def __init__(self, path: Optional[List[Cell]], explored: List[Cell]):
        self.path = path          # None if goal is unreachable
        self.explored = explored  # cells expanded, in order
        self.cost = len(path) - 1 if path else None

    @property
    def found(self) -> bool:
        return self.path is not None

def astar(maze: Maze, heuristic: str = "manhattan") -> SearchResult:
    """
    f(n) = g(n) + h(n)
    g = steps taken from start, h = estimated distance to goal.
    Returns the shortest path, or a result with path=None if unreachable.
    """
    h = HEURISTICS[heuristic]
    start, goal = maze.start, maze.goal

    counter = 0 # tie-breaker so the heap never compares cells
    open_heap = [(h(start, goal), 0, counter, start)]
    g_score: Dict[Cell, float] = {start: 0}
    came_from: Dict[Cell, Cell] = {}
    closed: Set[Cell] = set()
    explored: List[Cell] = []

    while open_heap:
        _, g, _, current = heapq.heappop(open_heap)
        if current in closed:
            continue # stale heap entry
        closed.add(current)
        explored.append(current)

        if current == goal:
            return SearchResult(_reconstruct(came_from, current), explored)

        for nb in maze.neighbors(current):
            tentative = g + 1
            if tentative < g_score.get(nb, math.inf):
                g_score[nb] = tentative
                came_from[nb] = current
                counter += 1
                heapq.heappush(open_heap, (tentative + h(nb, goal), tentative, counter, nb))

    return SearchResult(None, explored) # open set exhausted -> unreachable

def _reconstruct(came_from: Dict[Cell, Cell], current: Cell) -> List[Cell]:
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    return path[::-1]

# -----
# 4. Visualisation (console + optional matplotlib)
# -----
def render_console(maze: Maze, result: SearchResult) -> str:
    path = set(result.path or [])
    explored = set(result.explored)
    out = []
    for r in range(maze.rows):
        line = []
        for c in range(maze.cols):
            cell, ch = (r, c), maze.grid[r][c]
            if ch in "SG#":
                line.append(ch)
            elif cell in path:
                line.append("*")
            elif cell in explored:
                line.append(".")
            else:
                line.append(" ")
        out.append("".join(line))
    return "\n".join(out)

```

```

DEFAULT_MAZE = """
#####
#S....#.....#
#.#.#.#.#####.#
#.#....#.#.....#.#
#.#.###.#.#####.#
#.#....#.#...#.#
#.#####.#.#.#.#
#.....#.#...#.#
#.#####.#####.#
#.....#...G#
#####
"""

maze = Maze(DEFAULT_MAZE)
result = astar(maze, "manhattan")
print(render_console(maze, result))
print(f"\nPath length: {result.cost} steps | Nodes explored: {len(result.explored)}")

```

Path length: 39 steps | Nodes explored: 91

- ✓ Demo 2 – Compare heuristics

```
manhattan -> cost 39, explored 91 nodes
euclidean -> cost 39, explored 91 nodes
```

```
blocked = Maze("""
S.#...
..#.#.
###.#G.
""")
r = astar(blocked)
print(render_console(blocked, r))
print("\nReachable?", r.found) # False
```

```
S.#  
..# ##  
### G  
  
Reachable? False
```

▼ Demo 4 – Plot of search and final path

```
plot_matplotlib(maze, astar(maze, "manhattan"), "A* Maze Solver (Manhattan)")
```

